



Architecting Resilience: A Deep Dive into Enterprise-Grade Wazuh SIEM with Advanced Load Balancing and High-Availability

Author: Abu Saeid

Email: contact.abusaeid598@gmail.com

Phone: +8801752791256

Abstract

In the contemporary threat landscape, a Security Information and Event Management (SIEM) solution must offer not only comprehensive detection capabilities but also uncompromising resilience and scalability. Traditional SIEM deployments often struggle to maintain performance and availability under the massive data ingestion requirements of a large enterprise. This article details the successful design, implementation, and validation of an enterprise-grade Wazuh SIEM architecture engineered for maximum uptime and throughput. The core achievement lies in the strategic integration of advanced load balancing techniques, specifically leveraging Nginx Stream, HAProxy, and Keepalived, to implement both Active-Passive and Active-Active failover mechanisms across the entire security stack. This deployment model ensures that the system can handle massive agent loads and seamlessly survive component failures without data loss or service interruption.

1. The Foundation of Resilience: Enterprise Wazuh SIEM Architecture

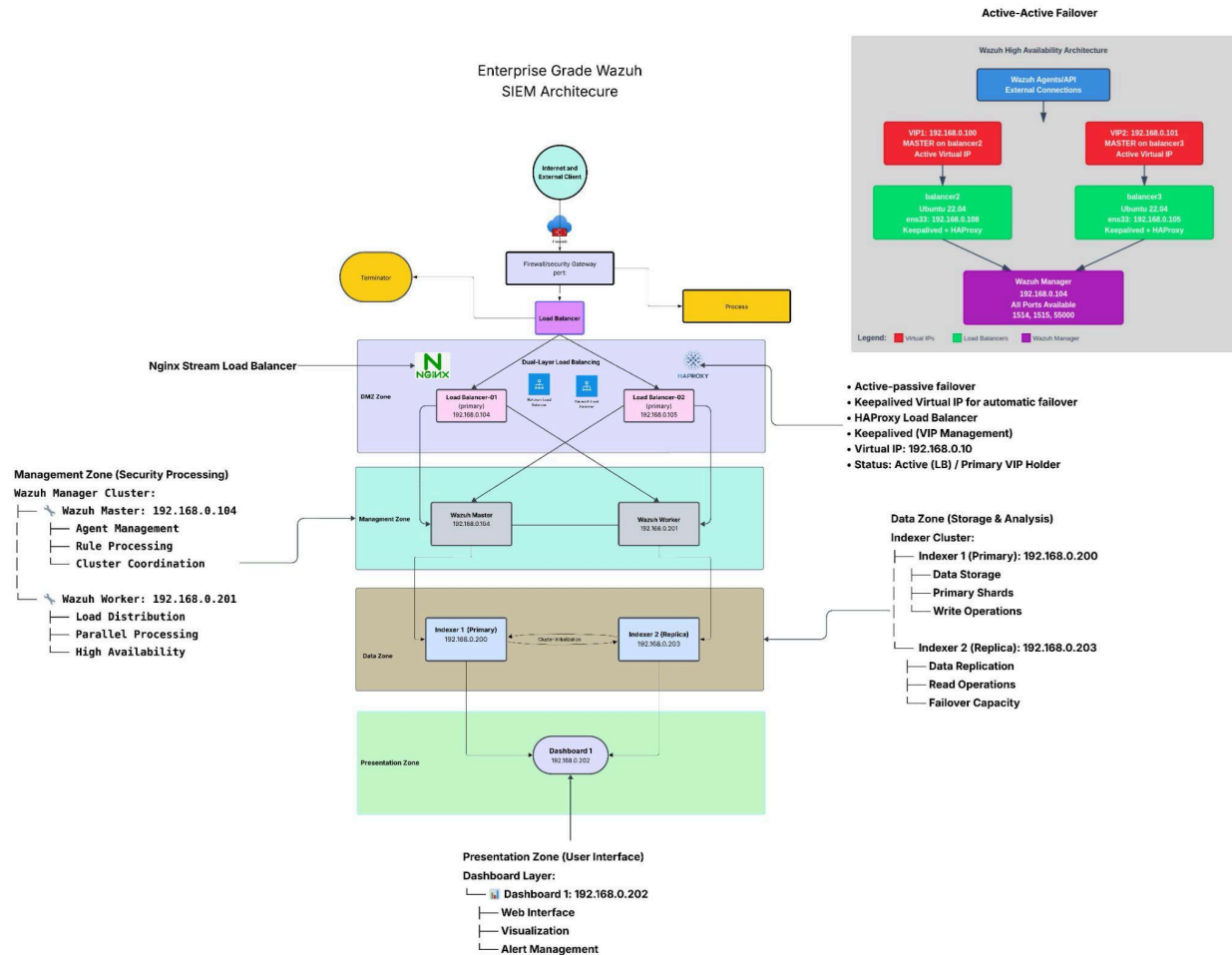


Fig: Main Architecture

The architectural design was predicated on the principle of decoupling services and establishing clear, segmented zones to enhance security, manageability, and scalability. The entire deployment is logically divided into four distinct zones, as illustrated in the initial architectural diagram:

1. DMZ Zone (Perimeter): Houses the load balancers, acting as the single point of entry for all external agent communication.

2. Management Zone (Processing): Contains the Wazuh Manager cluster (Master and Workers) responsible for agent registration, rule processing, and alert generation.

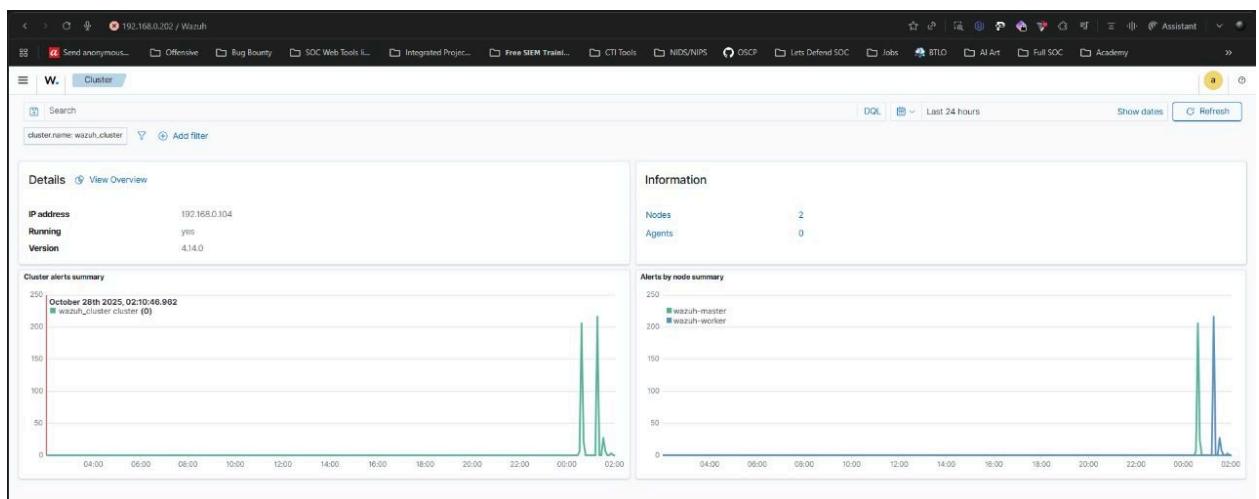
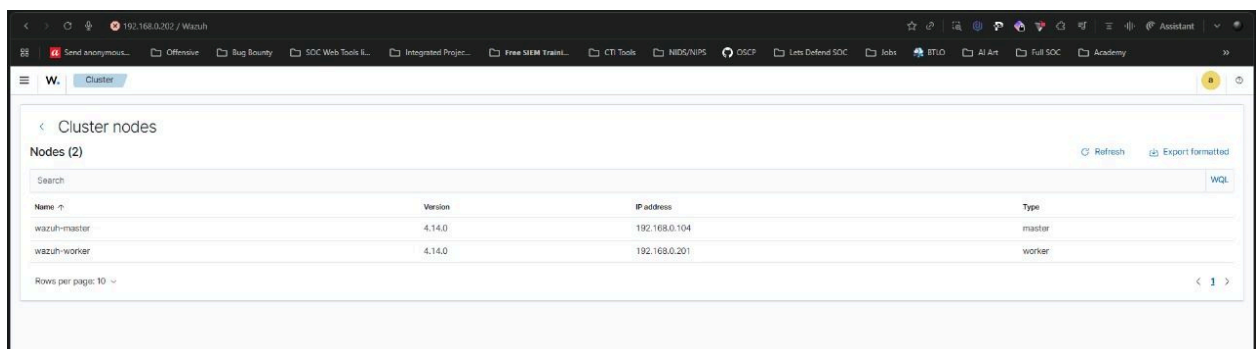
3.Data Zone (Storage & Analysis): Comprises the Indexer cluster, which handles the storage, indexing, and retrieval of security events.

4.Presentation Zone (User Interface): Hosts the Wazuh Dashboard for visualization, analysis, and alert management.

1.1 Core Component Configuration

The Wazuh cluster itself was configured for high availability and distributed processing:

- Wazuh Manager Cluster: A Master Node (192.168.0.104) was designated for cluster coordination and centralized configuration management, while a Worker Node (192.168.0.201) was deployed to handle the bulk of agent traffic and rule processing. This configuration, verified by the `ossec.conf` cluster settings and the successful installation of the worker manager, forms the basis of the Active-Active processing capability.



```

<cluster>
  <name>wazuh_cluster</name>
  <node_name>wazuh-master</node_name>
  <node_type>master</node_type>
  <key>eaff9931d09424be8fec0e2c4978fce1</key>
  <port>1516</port>
  <bind_addr>0.0.0.0</bind_addr>
  <nodes>
    <node>192.168.0.104</node>
    <node>192.168.0.201</node>
  </nodes>
  <hidden>no</hidden>
  <disabled>no</disabled>
</cluster>

</ossec_config>

```

The screenshot shows a terminal window with a dark background. At the top, there are tabs for 'Vaults', 'SFTP', and three IP addresses: '192.168.0.104', '192.168.0.200', and '192.168.0.203'. The terminal prompt is 'GNU nano 8.6' and the file being edited is '/etc/filebeat/filebeat.yml'. The file content is as follows:

```

# Wazuh - Filebeat configuration file
output.elasticsearch:
  protocol: https
  username: ${username}
  password: ${password}
  ssl.certificate_authorities:
    - /etc/filebeat/certs/root-ca.pem
  ssl.certificate: "/etc/filebeat/certs/wazuh-master.pem"
  ssl.key: "/etc/filebeat/certs/wazuh-master-key.pem"
setup.template.json.enabled: true
setup.template.json.path: '/etc/filebeat/wazuh-template.json'
setup.template.json.name: 'wazuh'
setup.ilm.overwrite: true
setup.ilm.enabled: false

filebeat.modules:
- module: wazuh
  alerts:
    enabled: true
  archives:
    enabled: false

logging.level: info
logging.to_files: true
logging.files:
  path: /var/log/filebeat
  name: filebeat
  keepfiles: 7
  permissions: 0644

logging.metrics.enabled: false

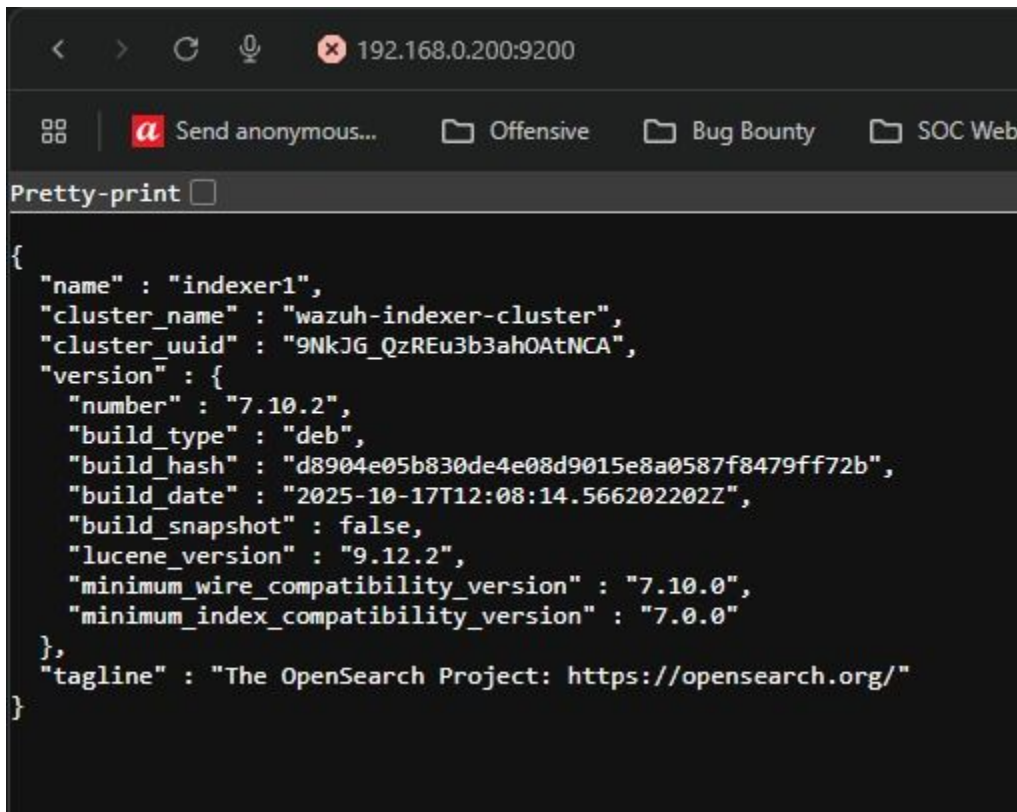
seccomp:
  default_action: allow
  syscalls:
    - action: allow
      names:
        - rseq

output.elasticsearch.hosts:
  - 192.168.0.200:9200
  - 192.168.0.203:9200

```

- Indexer Cluster: The data layer was secured with a minimum of two nodes: a Primary Indexer (192.168.0.200) and a Replica Indexer (192.168.0.203). This setup ensures data redundancy through replication, allowing the system to sustain the loss of a storage node without impacting data integrity or read operations.

The successful initialization and running status of both the Wazuh cluster and the Indexer cluster were confirmed, establishing a robust, multi-node foundation for the SIEM platform.



```
{
  "name" : "indexer1",
  "cluster_name" : "wazuh-indexer-cluster",
  "cluster_uuid" : "9NkJG_QzREu3b3ah0AtNCA",
  "version" : {
    "number" : "7.10.2",
    "build_type" : "deb",
    "build_hash" : "d8904e05b830de4e08d9015e8a0587f8479ff72b",
    "build_date" : "2025-10-17T12:08:14.566202202Z",
    "build_snapshot" : false,
    "lucene_version" : "9.12.2",
    "minimum_wire_compatibility_version" : "7.10.0",
    "minimum_index_compatibility_version" : "7.0.0"
  },
  "tagline" : "The OpenSearch Project: https://opensearch.org/"
}
```

```
< > ↻ 🔊 192.168.0.203:9200

🗄️ |  Send anonymous... 📁 Offensive 📁 Bug Bounty 📁 SOC Web Tools li...

Pretty-print ☐

{
  "name" : "indexer2",
  "cluster_name" : "wazuh-indexer-cluster",
  "cluster_uuid" : "9NkJG_QzREu3b3ahOAtNCA",
  "version" : {
    "number" : "7.10.2",
    "build_type" : "deb",
    "build_hash" : "d8904e05b830de4e08d9015e8a0587f8479ff72b",
    "build_date" : "2025-10-17T12:08:14.566202202Z",
    "build_snapshot" : false,
    "lucene_version" : "9.12.2",
    "minimum_wire_compatibility_version" : "7.10.0",
    "minimum_index_compatibility_version" : "7.0.0"
  },
  "tagline" : "The OpenSearch Project: https://opensearch.org/"
}
```

```
<indexer>
  <enabled>yes</enabled>
  <hosts>
    <host>https://192.168.0.203:9200</host>
    <host>https://192.168.0.200:9200</host>
  </hosts>
  <ssl>
    <certificate_authorities>
      <ca>/etc/filebeat/certs/root-ca.pem</ca>
    </certificate_authorities>
    <certificate>/etc/filebeat/certs/wazuh-master.pem</certificate>
    <key>/etc/filebeat/certs/wazuh-master-key.pem</key>
  </ssl>
</indexer>
```

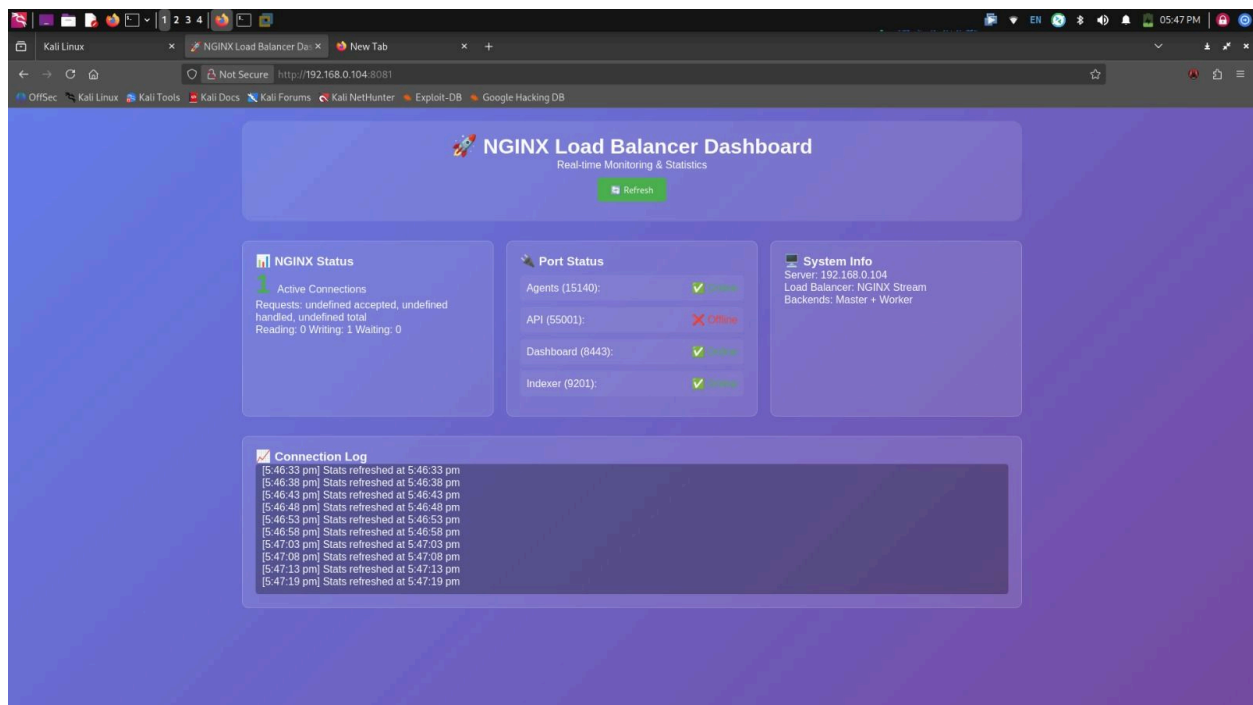
2. Mastering High Availability and Load Balancing

The true enterprise-grade nature of this deployment is defined by its sophisticated load balancing strategy, which employs a dual-layer approach to manage traffic flow and ensure service continuity.

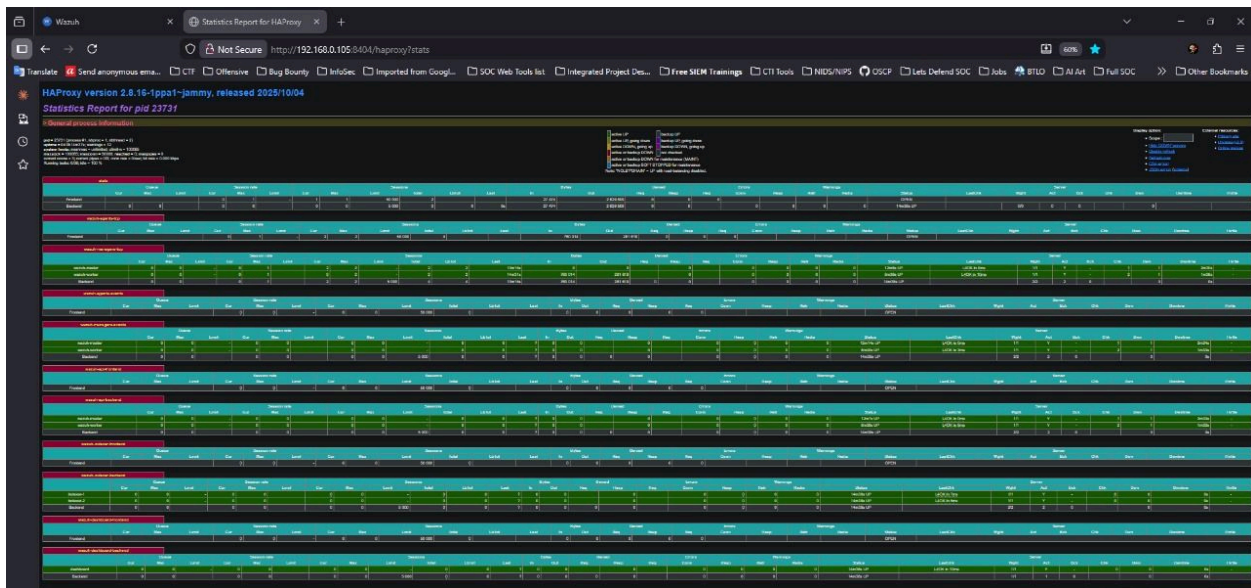
2.1 Dual-Layer Load Balancing Strategy

Agent communication is a high-volume, continuous stream of data, making the load balancer a critical component. We implemented a two-pronged strategy:

1.Nginx Stream Load Balancer: Used for simple, high-performance TCP stream forwarding, primarily for the initial Wazuh agent connection (port 1514). Its lightweight nature makes it ideal for raw data throughput.



2.HAProxy Load Balancer: Utilized for more intelligent Layer 4/7 load balancing, particularly for the Wazuh API and the Dashboard. HAProxy's advanced health checks and session persistence capabilities are essential for maintaining stateful connections.



2.2 Active-Passive Failover for Service Uptime

To eliminate the single point of failure inherent in any load balancer deployment, an Active-Passive failover mechanism was implemented using HAProxy and Keepalived.

Component	Role	Function
HAProxy	Primary Load Balancer	Distributes incoming agent traffic to the Wazuh Manager cluster.
Keepalived	High Availability Manager	Manages a Virtual IP (VIP) (192.168.0.10) that floats between two load balancer nodes.
Active-Passive Mode	Failover Strategy	The primary load balancer holds the VIP. If it fails, Keepalived automatically transfers the VIP to the passive (backup) load balancer, ensuring near-instantaneous and transparent service recovery.

The configuration files for this setup, which define the vrrp_instance and health check scripts, confirm the precise failover logic, ensuring that the agents always have a stable IP address to connect to, regardless of which physical load balancer node is active.

```
root@fancybear: /home/man: x + v
root@fancybear:/home/manager1# tail -f /var/ossec/logs/ossec.log
2025/11/03 11:07:43 wazuh-syscheckd: INFO: (6009): File integrity monitoring scan ended.
2025/11/03 11:07:43 wazuh-syscheckd: INFO: FIM sync module started.
2025/11/03 11:07:44 wazuh-modulesd:syscollector: INFO: Evaluation finished.
2025/11/03 11:07:45 wazuh-logcollector: INFO: (9203): Monitoring journal entries.
2025/11/03 11:07:47 wazuh-logcollector: INFO: (9204): 'Journald' timestamp was refreshed due to rotation.
2025/11/03 11:07:52 sca: INFO: Evaluation finished for policy '/var/ossec/ruleset/sca/cis_ubuntu22-04.yml'
2025/11/03 11:07:52 sca: INFO: Security Configuration Assessment scan finished. Duration: 9 seconds.
2025/11/03 11:08:08 rootcheck: INFO: Ending rootcheck scan.
2025/11/03 12:10:17 wazuh-modulesd:syscollector: INFO: Starting evaluation.
2025/11/03 12:10:27 wazuh-modulesd:syscollector: INFO: Evaluation finished.
2025/11/03 12:34:50 wazuh-agentd: ERROR: (1137): Lost connection with manager. Setting lock.
2025/11/03 12:34:50 wazuh-agentd: INFO: Closing connection to server ([192.168.0.105]:1514/tcp).
2025/11/03 12:34:50 wazuh-agentd: INFO: Trying to connect to server ([192.168.0.105]:1514/tcp).
2025/11/03 12:34:58 wazuh-logcollector: WARNING: Process locked due to agent is offline. Waiting for connection...
2025/11/03 12:35:04 wazuh-agentd: INFO: Closing connection to server ([192.168.0.105]:1514/tcp).
2025/11/03 12:35:04 wazuh-agentd: INFO: Trying to connect to server ([192.168.0.105]:1514/tcp).
2025/11/03 12:35:04 wazuh-agentd: INFO: (4102): Connected to the server ([192.168.0.105]:1514/tcp).
2025/11/03 12:35:04 wazuh-agentd: INFO: Server responded. Releasing lock.
2025/11/03 12:35:08 wazuh-logcollector: INFO: Agent is now online. Process unlocked, continuing...
2025/11/03 12:36:33 wazuh-agentd: ERROR: (1137): Lost connection with manager. Setting lock.
2025/11/03 12:36:33 wazuh-agentd: INFO: Closing connection to server ([192.168.0.105]:1514/tcp).
2025/11/03 12:36:33 wazuh-agentd: INFO: Trying to connect to server ([192.168.0.105]:1514/tcp).
2025/11/03 12:36:33 wazuh-agentd: INFO: (4102): Connected to the server ([192.168.0.105]:1514/tcp).
2025/11/03 12:36:33 wazuh-agentd: INFO: Server responded. Releasing lock.
```

https://www.notion.so/High-Availability-Not-High-Throughput-Classic-Active-Passive-HAProxy-Failover-with-Keepalived-2aee9f213ce3801a858fe75d7b4f12df?source=copy_link

How to set up Active-Passive Failover

2.3 Active-Active Failover for Throughput

While the load balancer nodes operate in Active-Passive mode for high availability, the Wazuh Manager cluster operates in an Active-Active configuration to maximize processing throughput.

The load balancers distribute incoming agent connections across the available Wazuh Worker nodes using algorithms like leastconn or roundrobin. This horizontal scaling allows the system to handle thousands of agents concurrently. The successful configuration of the Active-Active load balancing, as evidenced by the configuration files and the load balancer statistics dashboards, demonstrates the system's ability to distribute load efficiently and scale processing power on demand.

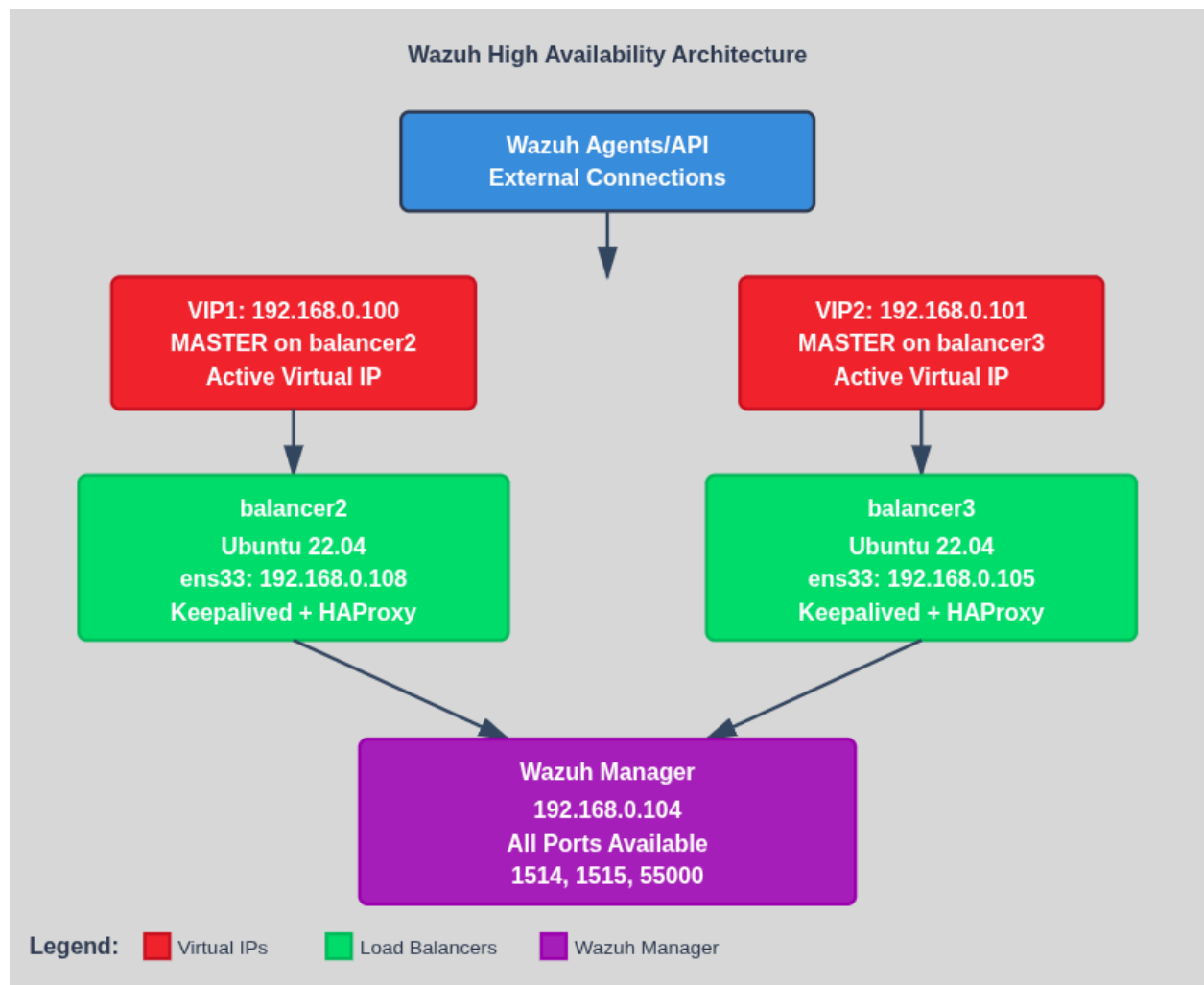


Fig: Active-Active Failover Diagram

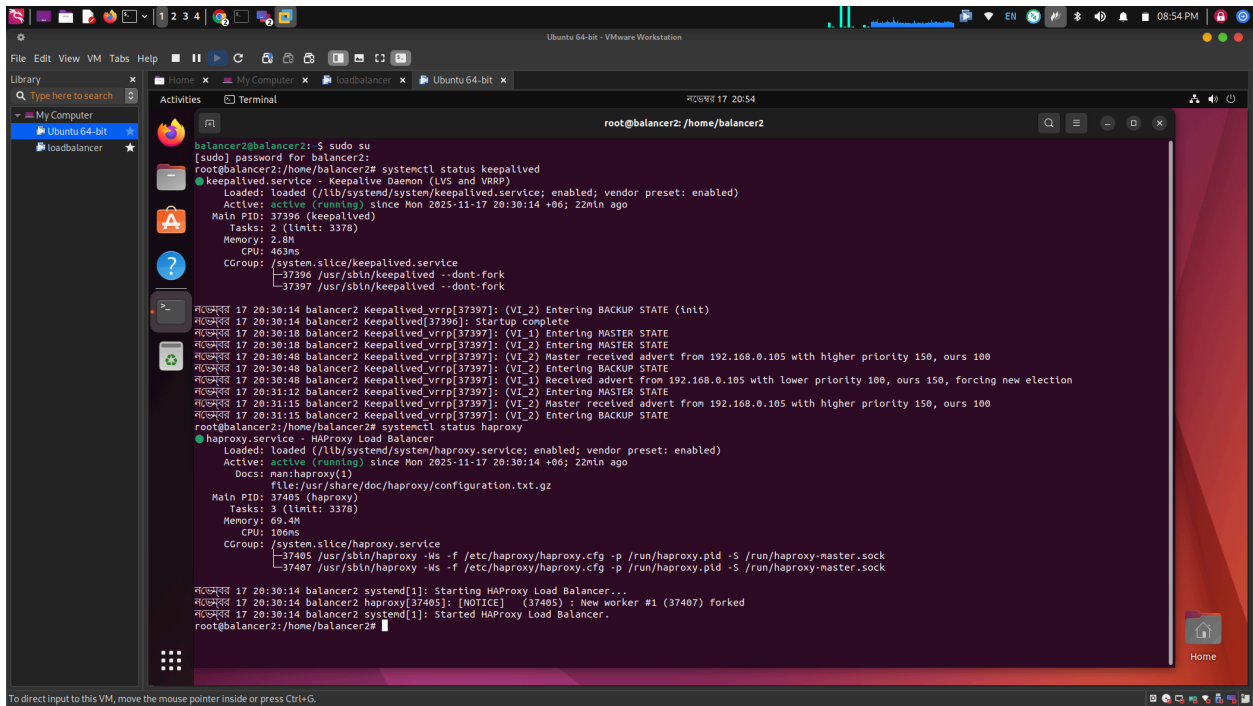
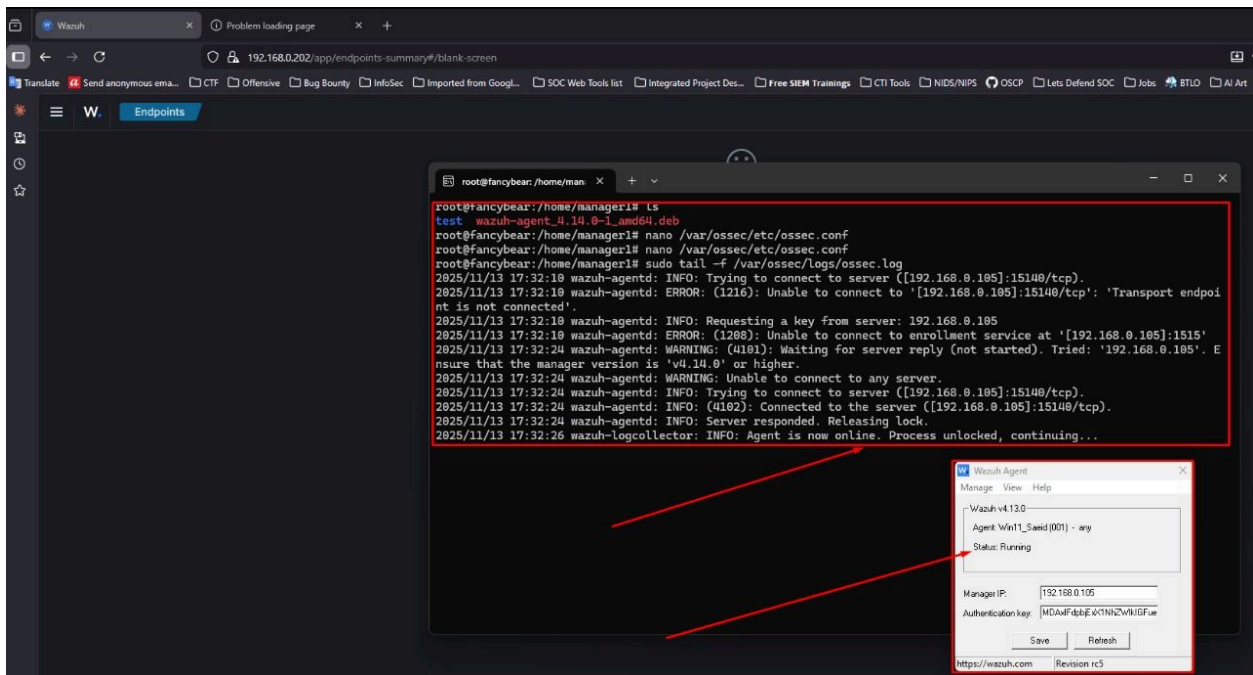


Fig: Active-Active VIP & keepAlived Failover

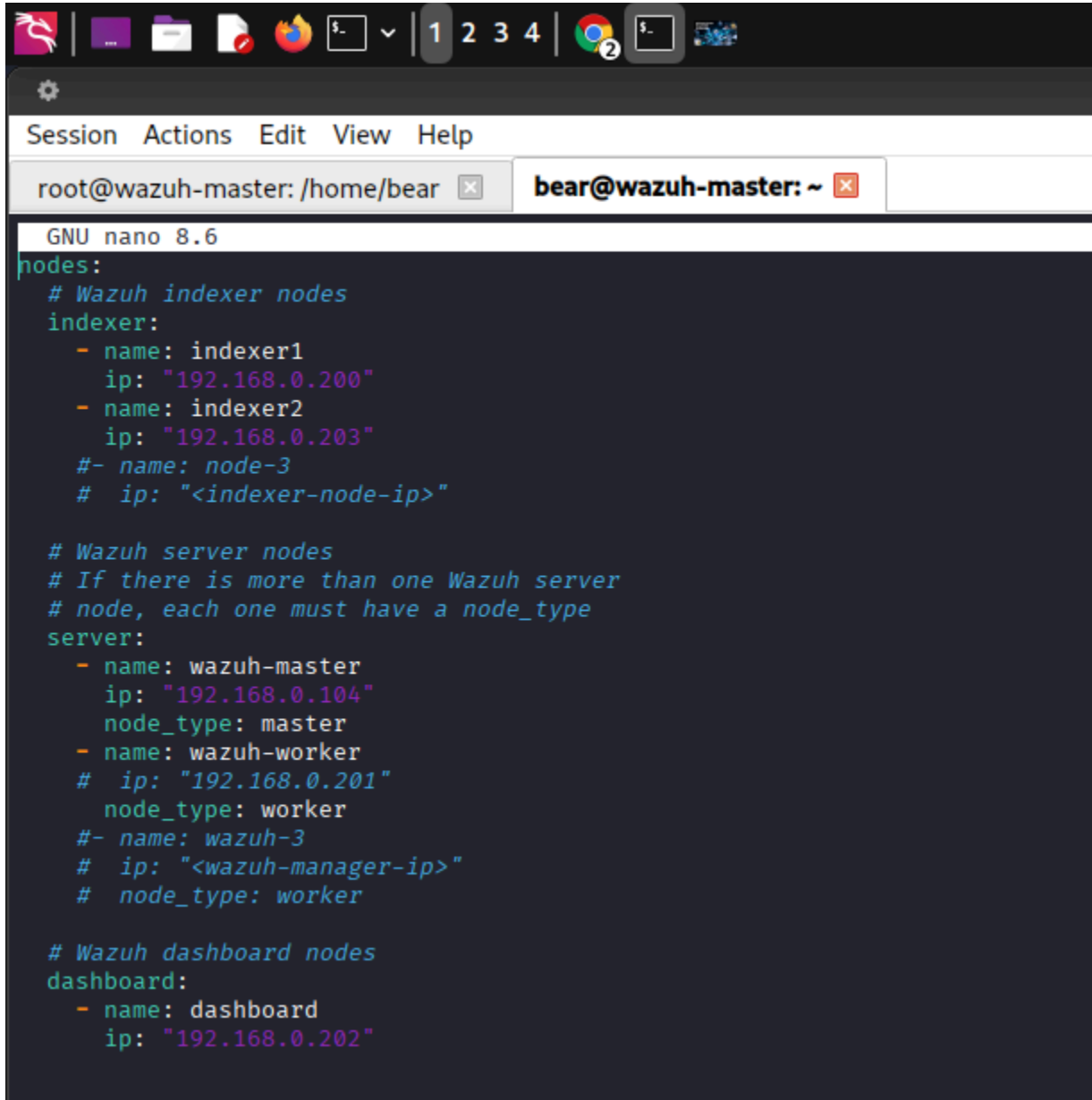


https://www.notion.so/Shared-Load-No-Theater-Active-Active-HAProxy-Load-Balancing-with-Dual-VIPs-and-Keepalived-2aee9f213ce380038684c54c4c462c20?source=copy_link

How to set up active-active Failover

3. Validation and Proof of Concept

The final phase of the project involved rigorous testing to validate the resilience and functionality of the complex architecture.

A screenshot of a Linux terminal window. The top bar shows a taskbar with icons for a terminal, file manager, and other applications. Below the taskbar is a menu bar with 'Session', 'Actions', 'Edit', 'View', and 'Help'. The terminal window has two tabs: 'root@wazuh-master: /home/bear' and 'bear@wazuh-master: ~'. The active tab is 'bear@wazuh-master: ~'. The terminal content shows the GNU nano 8.6 editor editing a file. The file content is a YAML configuration for Wazuh nodes. It defines indexer nodes (indexer1 and indexer2), server nodes (wazuh-master and wazuh-worker), and a dashboard node. The configuration includes names, IP addresses, and node types. Comments explain the purpose of each section and provide instructions for adding more nodes.

```
GNU nano 8.6
nodes:
# Wazuh indexer nodes
indexer:
  - name: indexer1
    ip: "192.168.0.200"
  - name: indexer2
    ip: "192.168.0.203"
#- name: node-3
# ip: "<indexer-node-ip>"

# Wazuh server nodes
# If there is more than one Wazuh server
# node, each one must have a node_type
server:
  - name: wazuh-master
    ip: "192.168.0.104"
    node_type: master
  - name: wazuh-worker
# ip: "192.168.0.201"
    node_type: worker
#- name: wazuh-3
# ip: "<wazuh-manager-ip>"
# node_type: worker

# Wazuh dashboard nodes
dashboard:
  - name: dashboard
    ip: "192.168.0.202"
```

Fig: config.yaml file

```

Vaults SFTP 192.168.0.104 192.168.0.104 192.168.0.104 192.168.0.104 ...
(root@wazuh-master)~[~bear]
# scp wazuh-install.sh wazuh-dashboard1@192.168.0.202:/home/wazuh-dashboard1
The authenticity of host '192.168.0.202 (192.168.0.202)' can't be established.
ED25519 key fingerprint is: SHA256:VD+Ise3X/YygXLPL0tKpwW0U3QW04bNAarfY0ZFZeQ0
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.0.202' (ED25519) to the list of known hosts.
wazuh-dashboard1@192.168.0.202's password:
wazuh-install.sh 100% 195KB 2.8MB/s 00:00

(root@wazuh-master)~[~bear]
# scp wazuh-install-files.tar wazuh-dashboard1@192.168.0.202:/home/wazuh-dashboard1
wazuh-dashboard1@192.168.0.202's password:
wazuh-install-files.tar 100% 14KB 975.3KB/s 00:00

(root@wazuh-master)~[~bear]
# cat config.yml
nodes:
  # Wazuh indexer nodes
  indexer:
    - name: indexer1
      ip: "192.168.0.200"
    - name: indexer2
      ip: "192.168.0.203"
    #- name: node-3
    # ip: "<indexer-node-ip>"

  # Wazuh server nodes
  # If there is more than one Wazuh server
  # node, each one must have a node_type
  server:
    - name: wazuh-master
      ip: "192.168.0.104"
      node_type: master
    - name: wazuh-worker
      # ip: "192.168.0.201"
      node_type: worker
    #- name: wazuh-3
    # ip: "<wazuh-manager-ip>"
    # node_type: worker

  # Wazuh dashboard nodes
  dashboard:
    - name: dashboard
      ip: "192.168.0.202"

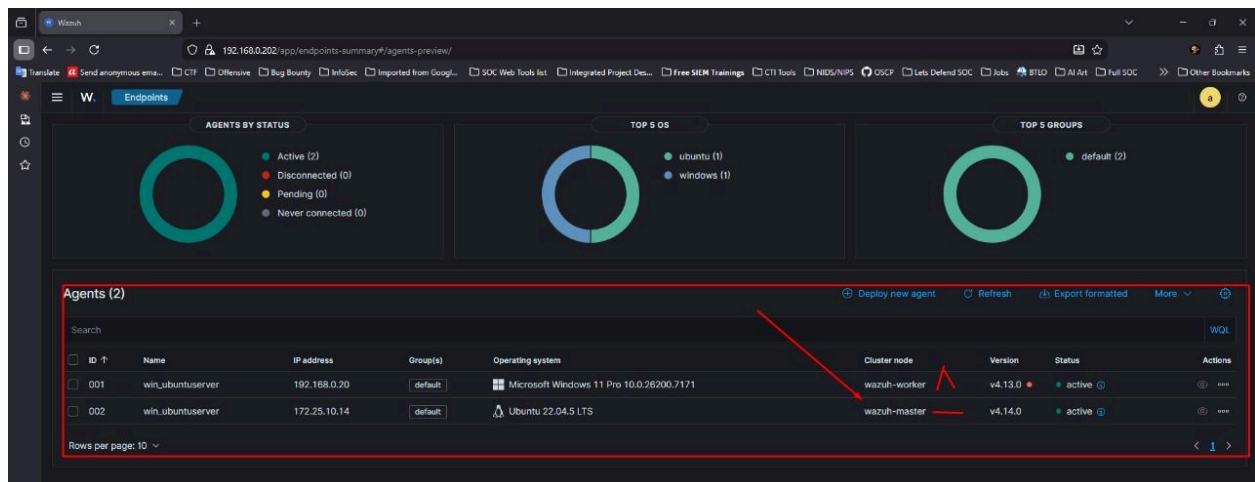
(root@wazuh-master)~[~bear]
#

root@dashboard: /home/waz X + -
GNU nano 6.2 /etc/wazuh-dashboard/opensearch_dashboards.yml
server.port: 443
opensearch.ssl.verificationMode: certificate
# opensearch.username: kibanaserver
# opensearch.password: kibanaserver
opensearch.requestHeadersAllowlist: ["securitytenant","Authorization"]
opensearch_security.multitenancy.enabled: false
opensearch_security.readonly_mode.roles: ["kibana_read_only"]
server.ssl.enabled: true
server.ssl.key: "/etc/wazuh-dashboard/certs/dashboard-key.pem"
server.ssl.certificate: "/etc/wazuh-dashboard/certs/dashboard.pem"
opensearch.ssl.certificateAuthorities: ["/etc/wazuh-dashboard/certs/root-ca.pem"]
uiSettings.overrides.defaultRoute: /app/wz-home
opensearch_security.cookie.secure: true
# Session expiration settings
opensearch_security.cookie.ttl: 900000
opensearch_security.session.ttl: 900000
opensearch_security.session.keepalive: true
server.host: 192.168.0.202
opensearch.hosts:
  - https://192.168.0.200:9200
  - https://192.168.0.203:9200

[ Read 21 lines ]
^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location
^X Exit ^R Read File ^_ Replace ^U Paste ^J Justify ^_ Go To Line
```

3.1 Agent Connectivity and Data Ingestion

To confirm end-to-end functionality, two agents were successfully deployed and connected: one on a Windows 11 endpoint and one on a Linux system. The agents successfully registered with the Wazuh cluster and began streaming security telemetry, which was then processed by the managers and indexed by the data cluster. This confirmed that the entire data pipeline, from the endpoint through the load balancers to the indexers, was fully operational.



3.2 The Ultimate Resilience Test: Zero-Downtime Failover

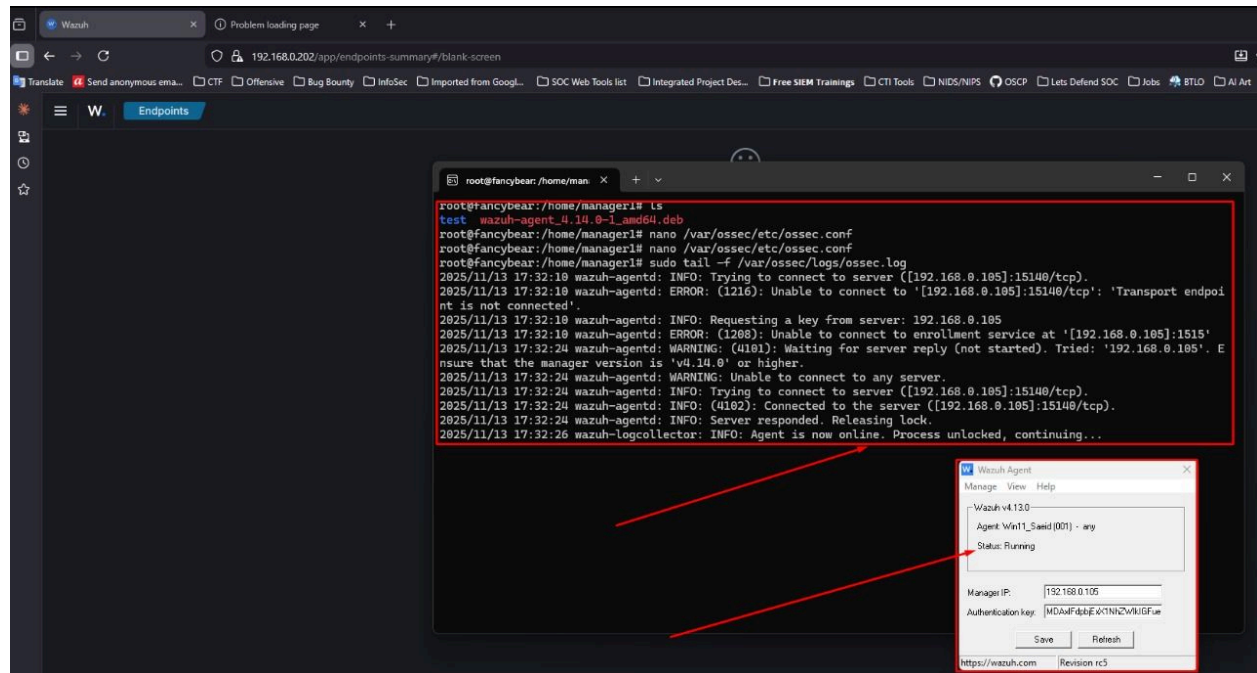
The most critical test involved simulating a failure of the primary processing node to validate the Active-Active failover mechanism within the Wazuh cluster.

Test Scenario: The Wazuh Master Node (192.168.0.104) was intentionally taken offline.

Observed Outcome: The agents, which were initially communicating with the cluster via the load balancer, did not lose connectivity. The load balancer, configured for Active-Active distribution across the available managers, immediately ceased routing new connections to the failed Master node and continued to direct all traffic to the remaining Wazuh Worker Node (192.168.0.201).

This successful failover, documented by the system logs and agent connection status, proves that the architecture achieves its goal: agent communication and data ingestion remain

uninterrupted even during a critical component failure. The system gracefully transitioned to a degraded, but fully functional, state, demonstrating true enterprise-grade resilience.



When I stop my wazuh-master, then the agent connected by 192.168.0.105 through HAProxy and Worker Node takes this responsibility also, when I stop the worker, then all traffic is handled by using the master.

Wazuh Version 4.14 focuses on **expanded IT hygiene endpoint inventory**, **dynamic agent configuration**, and **cloud security visibility**, bringing meaningful improvements through comprehensive asset auditing, streamlined operational flexibility, and a dedicated Microsoft Graph API integration.

Conclusion

The deployment of this enterprise-grade Wazuh SIEM architecture represents a significant step forward in building a truly resilient security monitoring platform. By strategically combining Active-Passive HA for the load balancing layer and Active-Active load distribution for the processing layer, the system is capable of delivering maximum throughput while guaranteeing service continuity. The successful validation of all load balancing configurations and the critical zero-downtime failover test confirms that this architecture is ready to meet the demanding performance and availability requirements of any large-scale enterprise environment. This project serves as a blueprint for organizations seeking to elevate their SIEM infrastructure to the highest standards of operational resilience.